



PagerDuty Connector — Credential Provisioning Guide

Version 0.1.0, 2026-09-14

Table of Contents

Overview.....	1
What kind of account to create.....	1
Permissions to grant	2
Generating the credential.....	2
Mapping values into Crogl.....	3
Verifying the setup.....	3
Reference documentation	4

Overview

The PagerDuty connector reads services, escalation policies, priorities, and on-call schedules, and creates and manages incidents, over PagerDuty's REST API v2. Its main purpose is **escalation**: when Crogl correlates alerts into a critical-priority finding, it can page the right on-call directly, without waiting on a human to notice — useful for a customer without 24/7 SOC coverage watching every alert.

It authenticates with two values sent on every call:

- A **REST API key**, sent as `Authorization: Token token=<key>`.
- An **acting user's email**, sent as the `From` header. PagerDuty requires this on every write (create/update an incident, add a note) to attribute the action to a real account user; it is ignored on reads.

These steps are performed by two people:

- A **PagerDuty Admin or Account Owner** creates the REST API key (only those two roles can) and confirms the acting user's role.
- Whoever administers your Crogl instance enters the resulting values when creating the connector.

You decide how much this connector can do mainly through which kind of API key you create (full-access vs. read-only — see below) and which user's email you supply as the acting user. Inside Crogl, `create-incident` fires immediately with no separate approval step—that is the intended behavior for unattended escalation, not a bug—while `update-incident` (acknowledge /resolve/reassign/reprioritize) and `add-comment` each require the operator to approve that specific action before it is sent.

IMPORTANT

There is **no delete, bulk, schedule-editing, or user/team-management** capability at any permission level—the connector's CLI has no verb that reaches those endpoints, regardless of what the API key could otherwise do.

What kind of account to create

PagerDuty offers two ways to generate a REST API v2 key:

- **General Access API Key** (account-wide)—created by an Admin or Account Owner under **Integrations → Developer Tools → API Access Keys**. Not tied to any one person's login, so it keeps working if that person leaves. **Recommended** for this connector.
- **User Token** (personal)—created by an individual user under their own **Profile → User Settings → API Access**. Scoped to "your own permissions"—if that person's account is later deactivated, the token stops working.

Either key type authenticates identically to this connector; General Access is recommended for the same reason a Jira Data Center or similar service account is recommended elsewhere in this repo — continuity independent of one person's employment status.

IMPORTANT

When creating a **General Access** key, PagerDuty's key-creation form offers a checkbox: **Read-only API Key**. Leave it **unchecked**. A read-only key makes every write verb (**create-incident**, **update-incident**, **add-comment**) fail with **403 Forbidden** while reads keep working — this is the single most likely first-setup mistake with this connector.

Separately from the API key, decide which **existing PagerDuty user's email** this connector will send as the **From** header on every write. This does not create a new account — it identifies whose name incidents/notes created by Crogl are attributed to. A dedicated, low-privilege user (e.g. **crogl-connector@yourcompany.com**) is recommended over a named individual's address, for the same continuity reason as above.

NOTE

(best-guess, not confirmed live) PagerDuty's documented account roles are Stakeholder, Limited User, User, Admin, and Account Owner. Stakeholder is read-only in the PagerDuty web UI (view incidents/services/schedules, no incident actions); Limited User and above can trigger/acknowledge/reassign/ resolve incidents. This guide assumes the REST API enforces the same role-based restriction on the **From** user that the web UI enforces, but that has not been confirmed against a live account for this guide — if writes fail with a permissions-shaped error even though the API key itself is full-access, check the **From** user's role first (Limited User or higher).

Permissions to grant

PagerDuty's REST API v2 key model does not expose fine-grained, per-endpoint scopes the way its OAuth2 apps do (OAuth2 scopes like **incidents.write** are a separate auth mechanism this connector does not use). A REST API key is effectively all-or-nothing at the **key** level:

Choice	What it enables
Full-access General Access or User Token key (Read-only unchecked)	Every command in the connector: list-*/get/query reads, plus create-incident , update-incident , add-comment .
Read-only API Key checked	Only the list-*/get/query reads. Every write verb returns 403 .

The **From** user's **role** is the other lever (see the note above) — grant it at least **Limited User** if you want writes to succeed, or **Stakeholder** if you deliberately want this connector limited to reads regardless of the API key's own access level.

There is no admin-consent or app-authorization step for a REST API v2 key (unlike an OAuth2 app) — the key is usable immediately once created.

Generating the credential

Performed while signed in as an **Admin or Account Owner**:

1. Go to **Integrations** → **Developer Tools** → **API Access Keys**.

2. Click **Create New API Key**.
3. Give it a description, e.g. **crogl-connector**.
4. Leave **Read-only API Key unchecked** (see the warning above) unless you deliberately want a read-only connector.
5. Click **Create Key**, then **copy the key value immediately** — PagerDuty shows the full key only once. Store it in your secret manager; you will paste it into Crogl in the next section.

Separately, confirm (or create) the PagerDuty user whose email you will use as the **From** value, and note that user's **role** (see above).

Mapping values into Crogl

One row per **connector.json** credential field:

Crogl field	Where it came from	Notes
api_key	The REST API key copied above	Secret. Sent as Authorization: Token token=<key> . Must not be a Read-only API Key if you want the write verbs to work.
from_email	The acting PagerDuty user's email address	Not actually sensitive data, but stored as a secret field because Crogl's connector schema requires every bound header to carry a secret value — see the connector's SKILL.md for the tradeoff (this field can't be viewed or edited after the connector is created, only replaced). Sent as the From header on writes only.

There is no host/tenant/instance identifier to supply — PagerDuty's REST API v2 base URL (<https://api.pagerduty.com>) is the same for every account.

Verifying the setup

After creating the connector instance in Crogl, run a read first (e.g. list services, or list incidents from the last week). Interpret the result:

- **Rows returned** — success.
- **No list-services results (0 rows)** (or similar) on a broad query — the call succeeded and simply matched nothing; confirm services/incidents actually exist in this PagerDuty account, or narrow/widen your filters.
- **401 Unauthorized** — the **api_key** is wrong, expired, or was revoked. Reissue it under **API Access Keys** and re-enter the value. (Crogl does not validate the key when the connector is created; a bad key first surfaces here.)
- **403 Forbidden on reads** — uncommon for a full-access key; check the key wasn't created as read-only, or that the account doesn't have a team-based access restriction excluding this key.

- **403 Forbidden only on writes, while reads work**—the strongest signal the API key is a **Read-only API Key**. Create a full-access key and update the connector's `api_key` field.
- **400 Bad Request on a write mentioning the requester**—the `from_email` value doesn't match a real, active PagerDuty user on this account, or that user's role doesn't permit the action (see the role note above). Confirm the email and the user's role.
- **404 Not Found**—a service/escalation-policy/incident id used in the request doesn't exist or isn't visible to this key. Not a setup problem with the credential itself.

If you intend to let this connector page on-call, verify `create-incident` once deliberately against a **test service** (create a throwaway PagerDuty service first) rather than a production one, and confirm the resulting incident is attributed to the `from_email` user you configured.

Reference documentation

The auth model, endpoint shapes, and roles in this guide and in the connector's `SKILL.md` were checked against these PagerDuty pages and its published OpenAPI spec, not recalled from training data:

- [PagerDuty REST API v2 OpenAPI spec](#) ([PagerDuty/api-schema, reference/REST/openapiv3.json](#))—the `Authorization: Token token=<key>` security scheme, the required `From` header on `POST /incidents`, `PUT /incidents/{id}`, and `POST /incidents/{id}/notes`, and the `service/escalation_policy/priority` reference-object shapes.
- [PagerDuty Events API v2 OpenAPI spec](#)—confirms the Events API v2 (a different, unbuilt connector's territory) is meant for raw monitoring-tool ingestion via a per-service routing key, and that PagerDuty's own guidance since 2017 recommends the REST API's synchronous Incidents API instead for a system-of-record caller like this one.
- [API Access Keys](#)—the **Integrations** → **Developer Tools** → **API Access Keys** menu path, the Admin/Account Owner role requirement to create a General Access key, the **Read-only API Key** option, and the personal **User Token** alternative under **Profile** → **User Settings** → **API Access**.
- [User Roles](#)—the Stakeholder / Limited User / User / Admin / Account Owner role definitions and which can trigger/acknowledge/reassign/resolve incidents, underlying this guide's (unconfirmed-live) note about the `From` user's role.
- [REST API Rate Limits](#)—the 960-requests-per-minute-per-token limit and the `ratelimit-*` response headers cited in the connector's `SKILL.md`.